

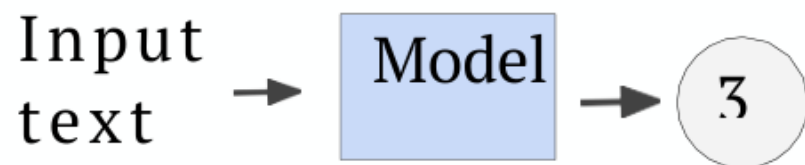
AIX

Historial Overview for NLP and Transformer

Mingyu Kim,
Mar, 31st, 2026

NLP tasks overview

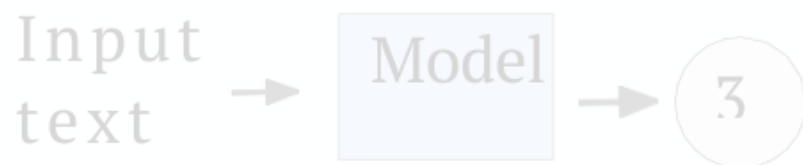
Classification



- Sentiment extraction
- Intent detection
- Language detection
- Topic modeling

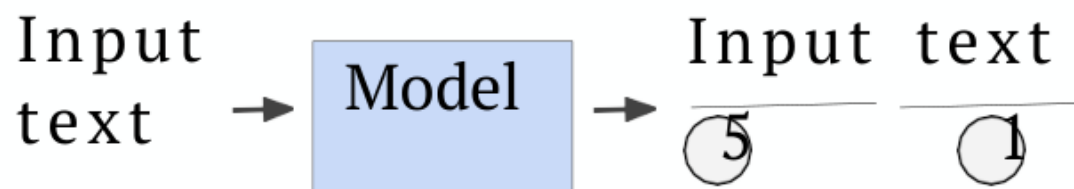
NLP tasks overview

Classification



- Sentiment extraction
- Intent detection
- Language detection
- Topic modeling

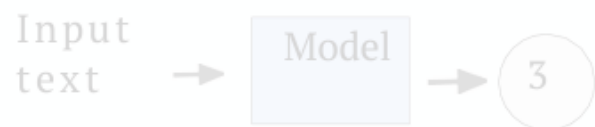
“Multi”-classification



- Part of speech tagging
- Named entity recognition
- Dependency parsing
- Constituency parsing

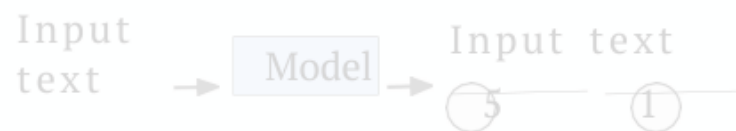
NLP tasks overview

Classification



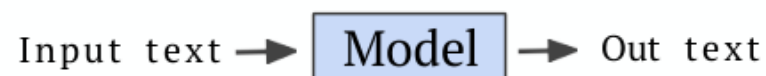
- Sentiment extraction
- Intent detection
- Language detection
- Topic modeling

“Multi”-classification



- Part of speech tagging
- Named entity recognition
- Dependency parsing
- Constituency parsing

Generation



- Machine translation
- Question answering
- Summarization
- Text generation

Sentimental Extraction



Datasets

 Amazon reviews

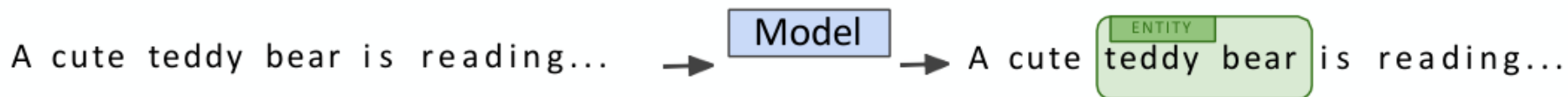
 IMDB critiques

 Twitter

Evaluation metrics

- Accuracy → % of observations that were correctly predicted?
- Precision → % of predicted positive that were correct?
- Recall → % of actually positive that were correct?
- F1 score → score that is a function of precision and recall

Named Entity Recognition



Datasets

 Annotated Reuters newspaper (CoNLL-2003, CoNLL++)

Evaluation metrics

- Accuracy
- Precision
- Recall
- F1 score

at a token level, per entity type

Machine Translation

A cute teddy bear is reading → Model → Un ours en peluche mignon lit

Datasets

  WMT'14 English-French

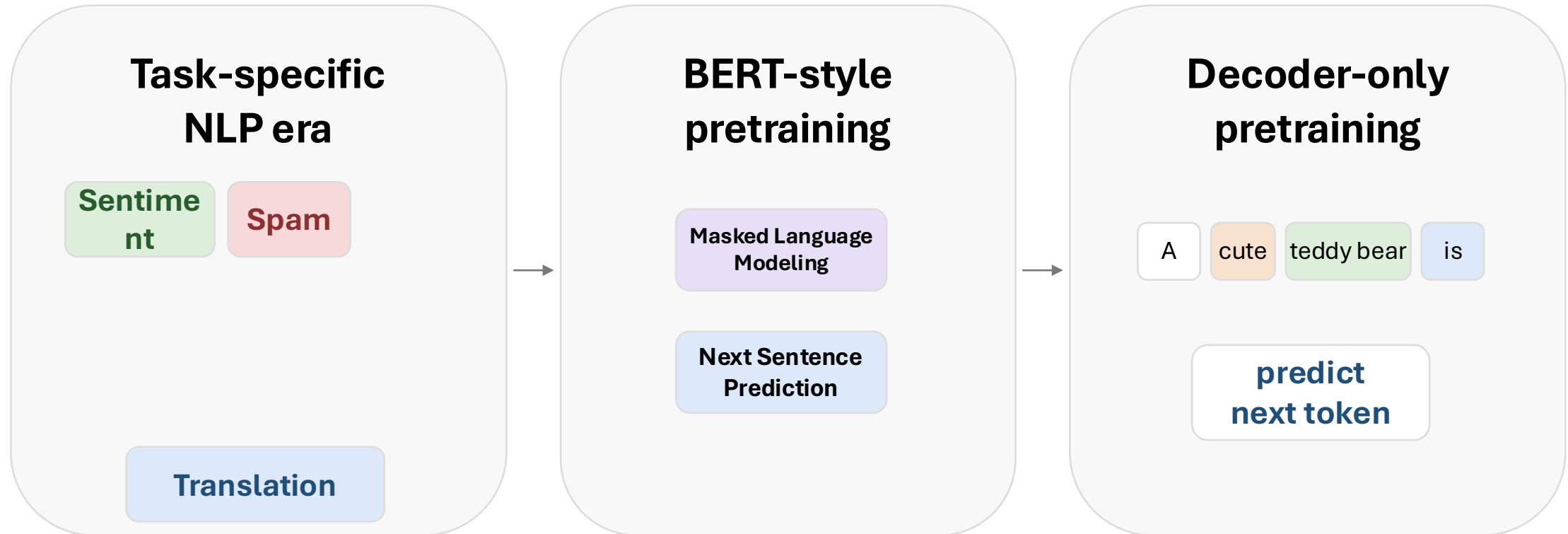
  WMT'14 English-German

Evaluation metrics

- BLEU → quality of text translated, similar to “precision”
- ROUGE → quality of text generated, similar to “recall”
- Perplexity → quantifies how ‘surprised’ the model is to see some words together

Task Examples and Pretraining Intuition

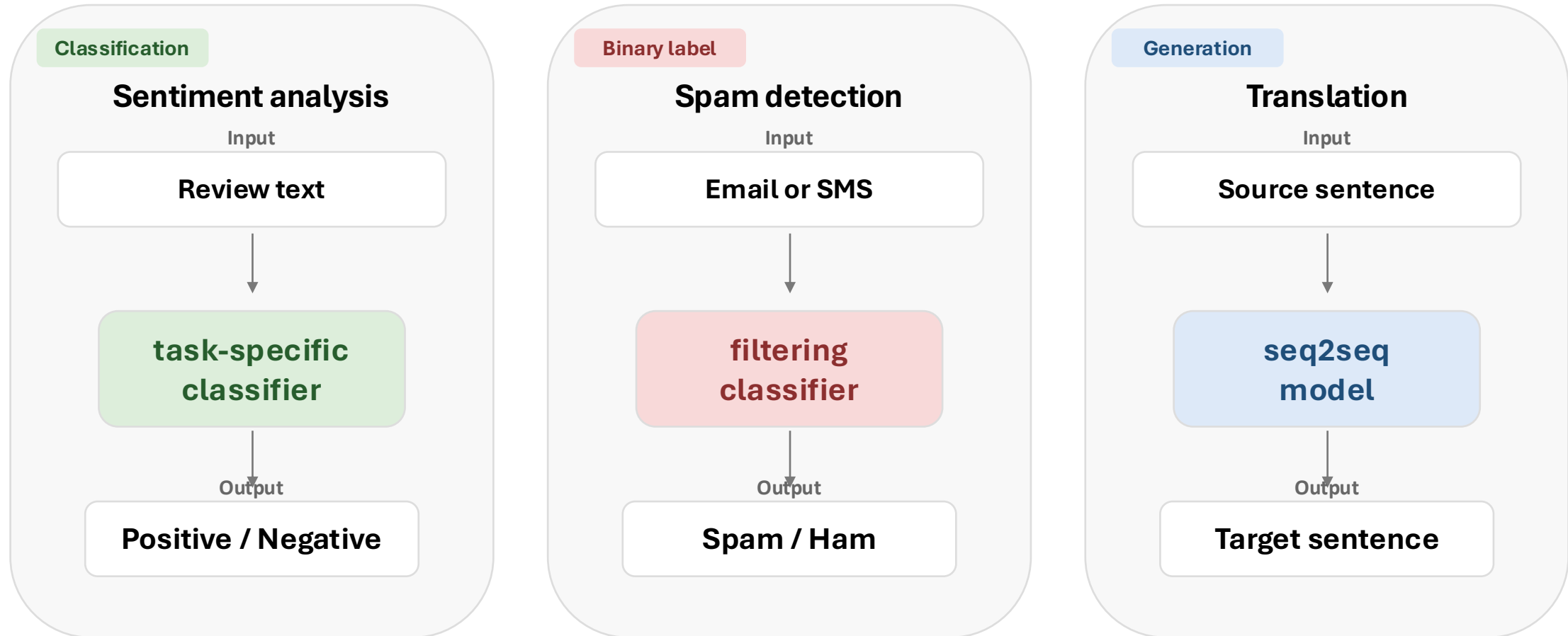
- This supplement adds concrete task examples for sentiment analysis, spam filtering, and translation.
- It also visualizes BERT's two proxy tasks and the loss used in next-token pretraining.
- The goal is to connect architecture, task type, and training objective with simple examples.



Concrete examples make the model families easier to remember.

Three Task Types Before General-Purpose LLMs

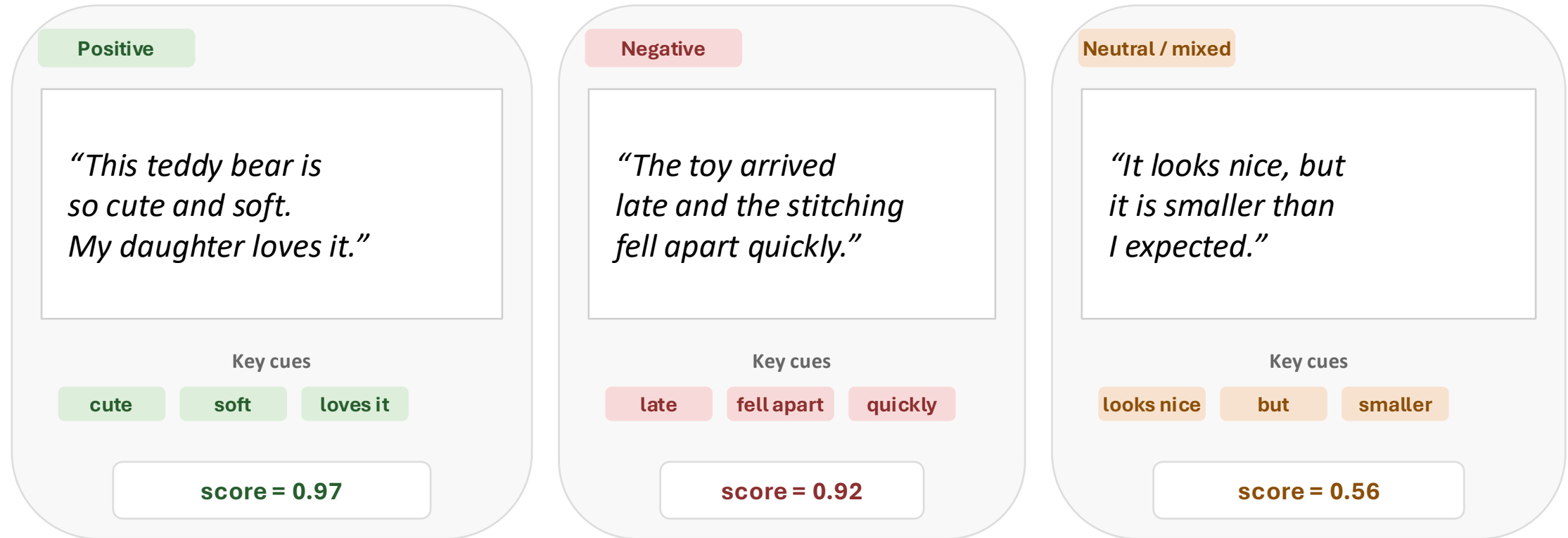
- Before foundation models, most NLP systems were trained for one task at a time.
- Sentiment and spam are classification tasks, while translation is a sequence generation task.
- The output format already tells us which model family is likely to work best.



Different tasks ask the model to return a label, a score, or a new sentence.

Sentiment Analysis: Turn a Review into a Label

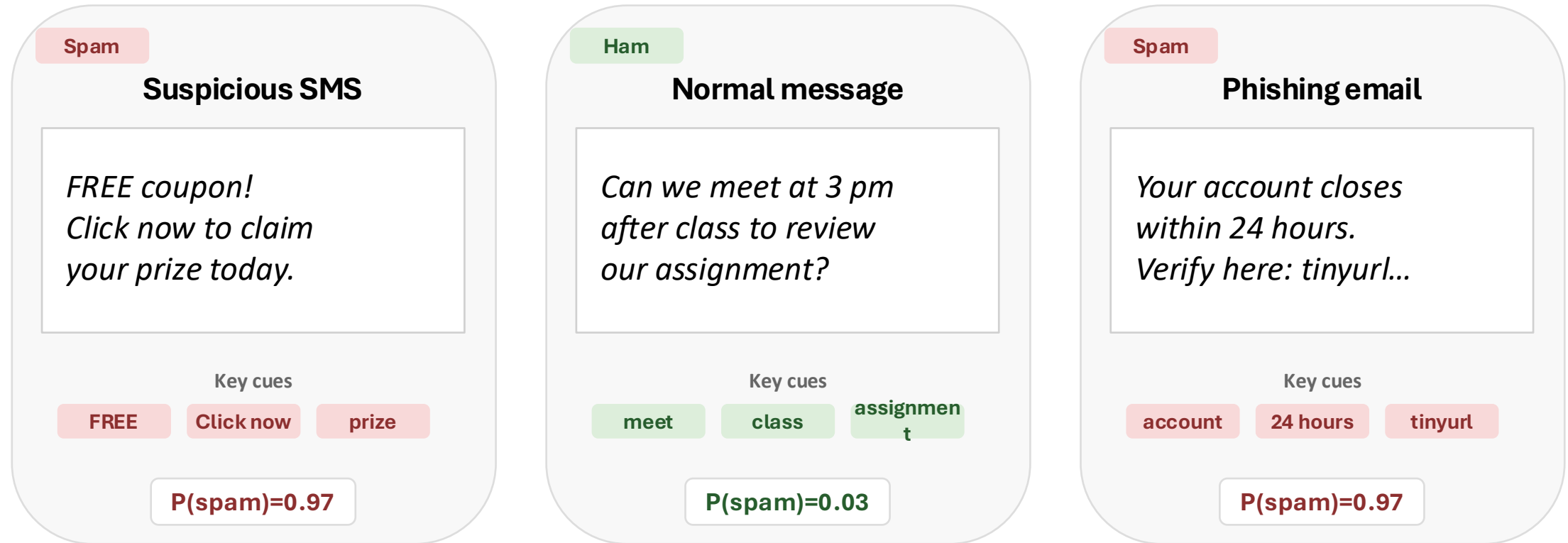
- The input is a sentence or review, and the output is usually a sentiment label or score.
- The model learns which words or phrases are strong positive or negative cues in context.
- Before decoder-only LLMs, this was a classic encoder-only or feature-based classification task.



Sentiment analysis predicts an opinion, not a new sentence.

Spam Detection: Filter Unwanted Messages

- Spam detection is usually a binary classification problem: spam or ham.
- Useful cues include suspicious words, URLs, urgent tone, sender patterns, and formatting.
- This task became a classic example of feature-based text classification long before modern LLMs.



A spam filter returns a yes/no decision rather than generating a new sentence.

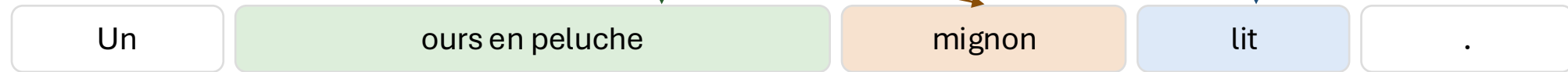
Machine Translation: Input Text and Output Text

- Translation is a generation task because the system must produce a new sequence in another language.
- The output can change word order, phrase grouping, and sentence length relative to the source.
- This is why translation needs a decoder or output model, not just a classifier head.

Source (English)



Target (French)



Different phrase grouping

Different word order

**Output sentence is generated
token by token**

Translation maps one sentence to another, so the output must be generated token by token.

Tokenization

A cute teddy bear is reading.

arbitrary

A	cute	teddy bear	is	reading	.
---	------	------------	----	---------	---

word

A	cute	teddy	bear	is	reading	.
---	------	-------	------	----	---------	---

sub-word

A	cute	ted	##dy	bear	is	read	##ing	.
---	------	-----	------	------	----	------	-------	---

Character Level

A	_	c	u	t	e	_	t	e	d	d	y	_	b	e	a	r	_	i	s	_	r	e	a	d	i	n	g	.
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

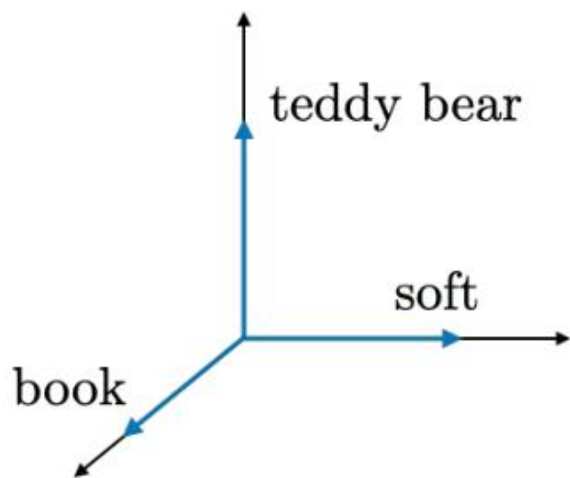
Tokenization Summary

Method	Pros	Cons
Word-level	• Simple • Interpretable	• Risk of OOV • Does not leverage knowledge of root
Subword-level e.g. WordPiece, BPE	• Leverages common prefixes and suffixes • Learned from the data	• Risk of OOV, though less than word-level
Character-level	• Small chance of OOV • Robust to Casing and Misspellings	• Makes computations slower • Embeddings not interpretable

Token Representation

Motivation

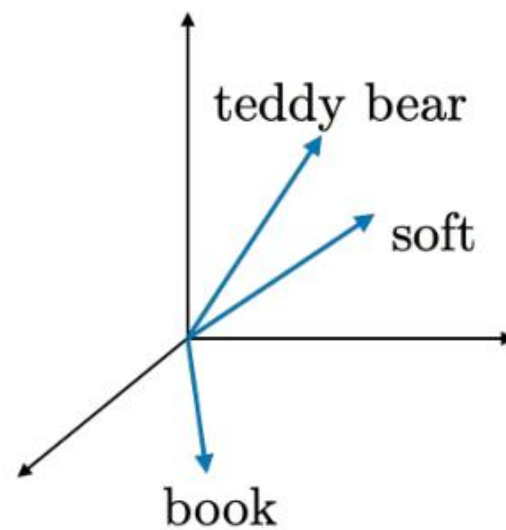
Naive (one-hot) encoding



$$\text{soft} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \begin{aligned} \langle \text{teddy bear}, \text{book} \rangle &= 0 \\ \langle \text{teddy bear}, \text{soft} \rangle &= 0 \end{aligned}$$



Learned embedding



$$\text{soft} = \begin{pmatrix} 0.95 \\ 0.32 \\ 0.01 \end{pmatrix} \quad \begin{aligned} \langle \text{teddy bear}, \text{book} \rangle &\sim 0 \\ \langle \text{teddy bear}, \text{soft} \rangle &\sim 1 \end{aligned}$$

Word2Vec

Overview

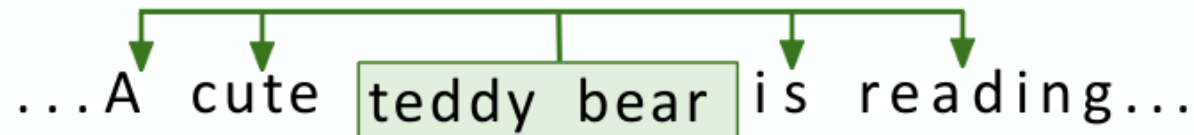
- Neural network with a **proxy task** over billions of words worth of text
- Learns an embedding layer

Proxy tasks

- CBOW (continuous bag of words)



- Skip-Gram



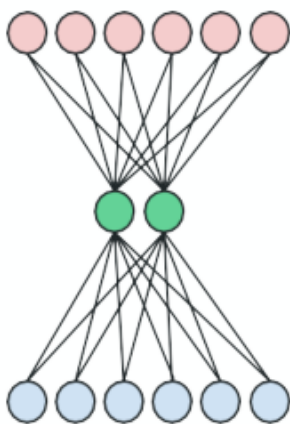
Word2Vec

Architecture

output

hidden

input



size V

size d

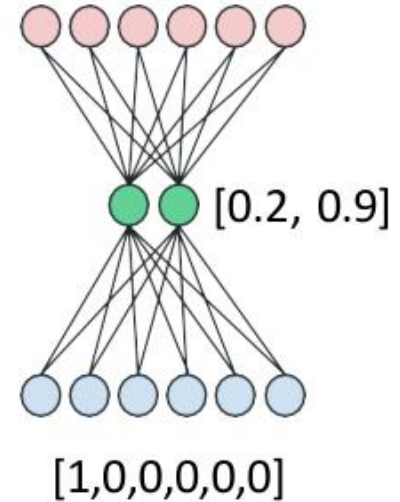
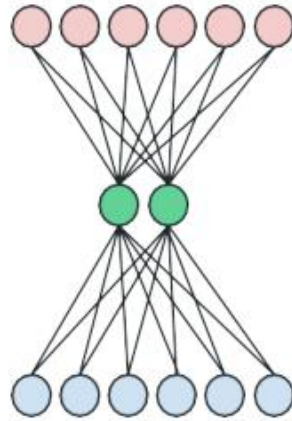
size V

Word2Vec

Example with predicting next word

output : A cute teddy bear is reading

A cute teddy bear is reading



input : A cute teddy bear is reading

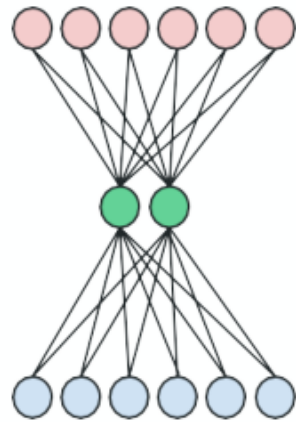
A cute teddy bear is reading

Word2Vec

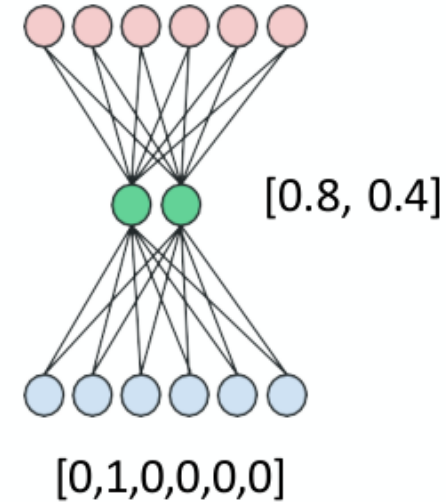
Example with predicting next word

output : A cute teddy bear is reading

A cute teddy bear is reading



[0.2, 0.2, 0.2, 0.1, 0.2, 0.1]



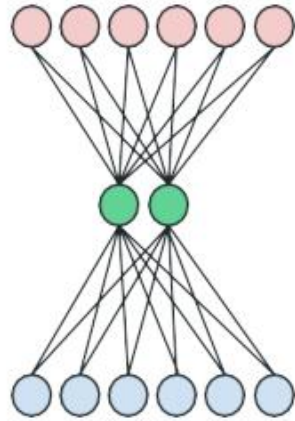
input : A cute teddy bear is reading

A cute teddy bear is reading

Word2Vec

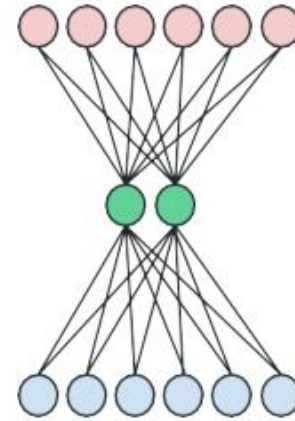
Example with predicting next word

A cute teddy bear **is** reading



A cute **teddy bear** is reading

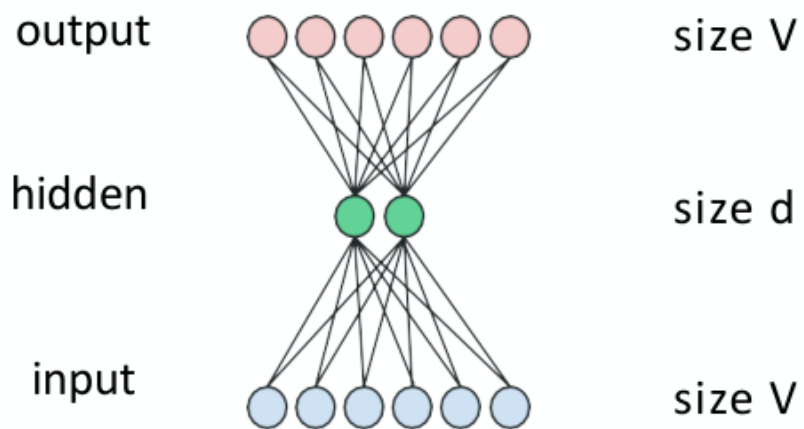
A cute teddy bear is **reading**



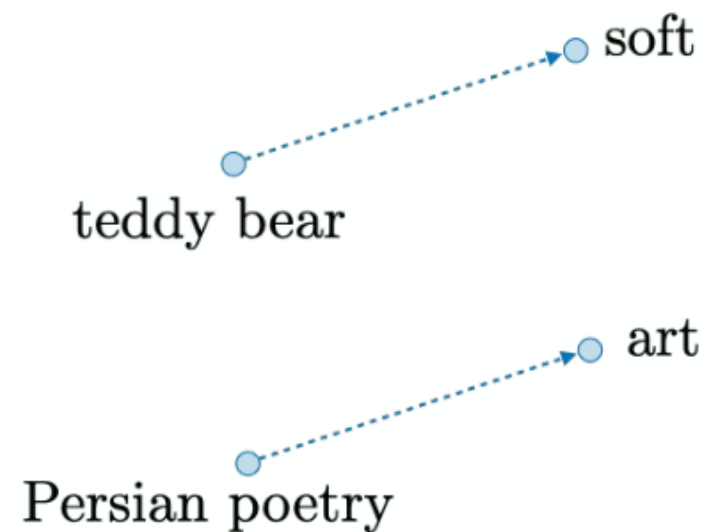
A cute teddy bear **is** reading

Word2Vec

A cute teddy bear is reading



A cute teddy bear is reading

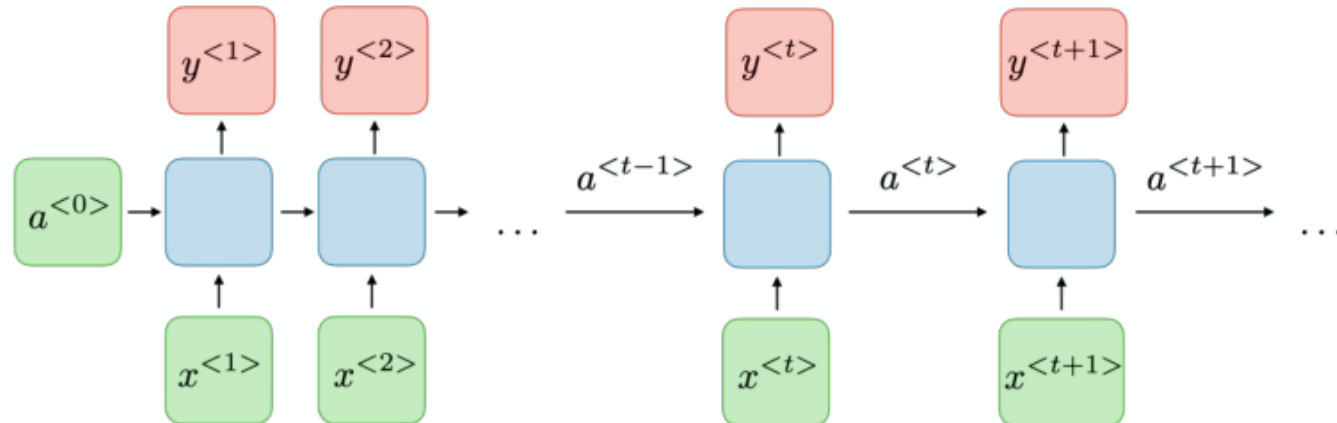


Recurrent Neural Networks

Overview

- First introduced in the 80s
- Class of neural networks where connections form a temporal sequence

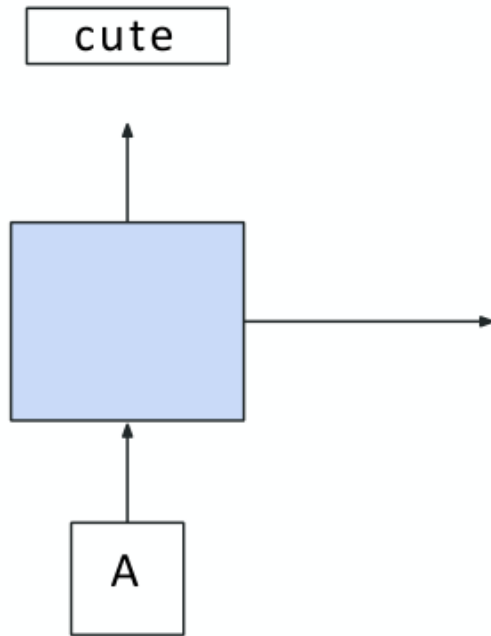
General form



Recurrent Neural Networks

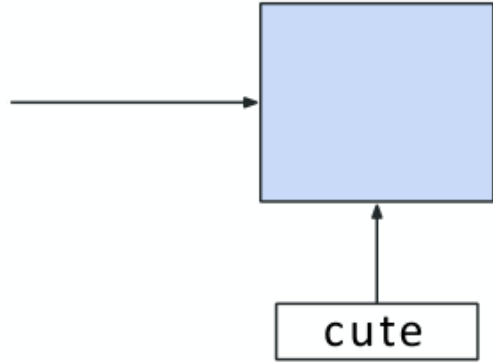
A

Recurrent Neural Networks



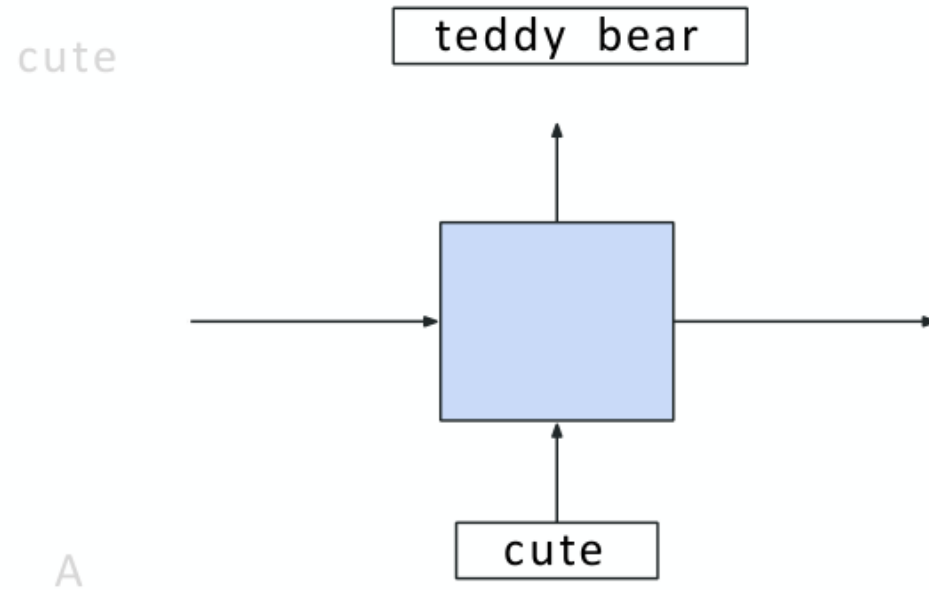
Recurrent Neural Networks

cute



A

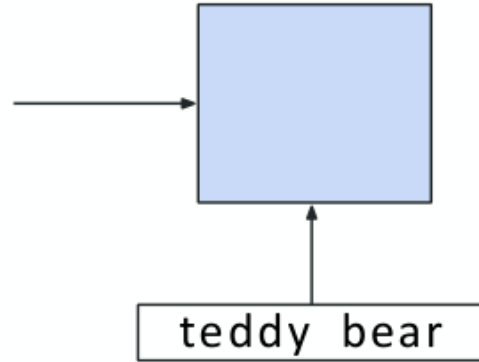
Recurrent Neural Networks



Recurrent Neural Networks

cute

teddy bear

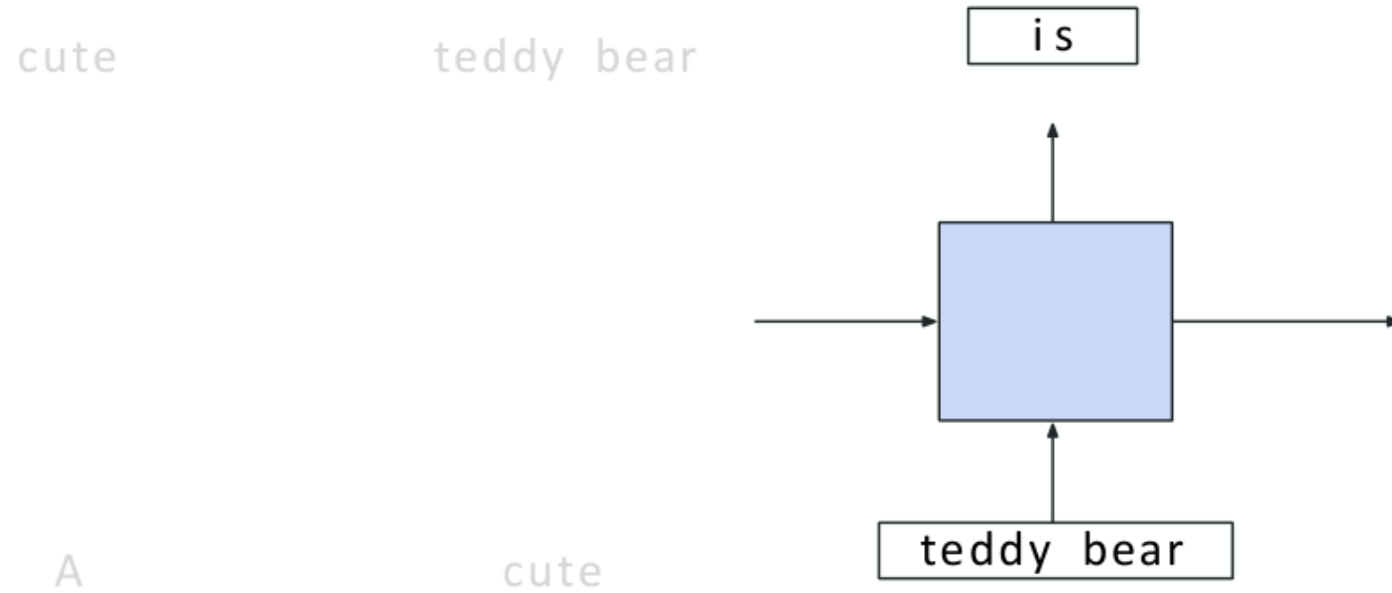


A

cute

teddy bear

Recurrent Neural Networks

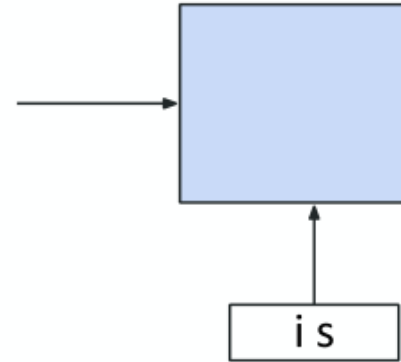


Recurrent Neural Networks

cute

teddy bear

is



A

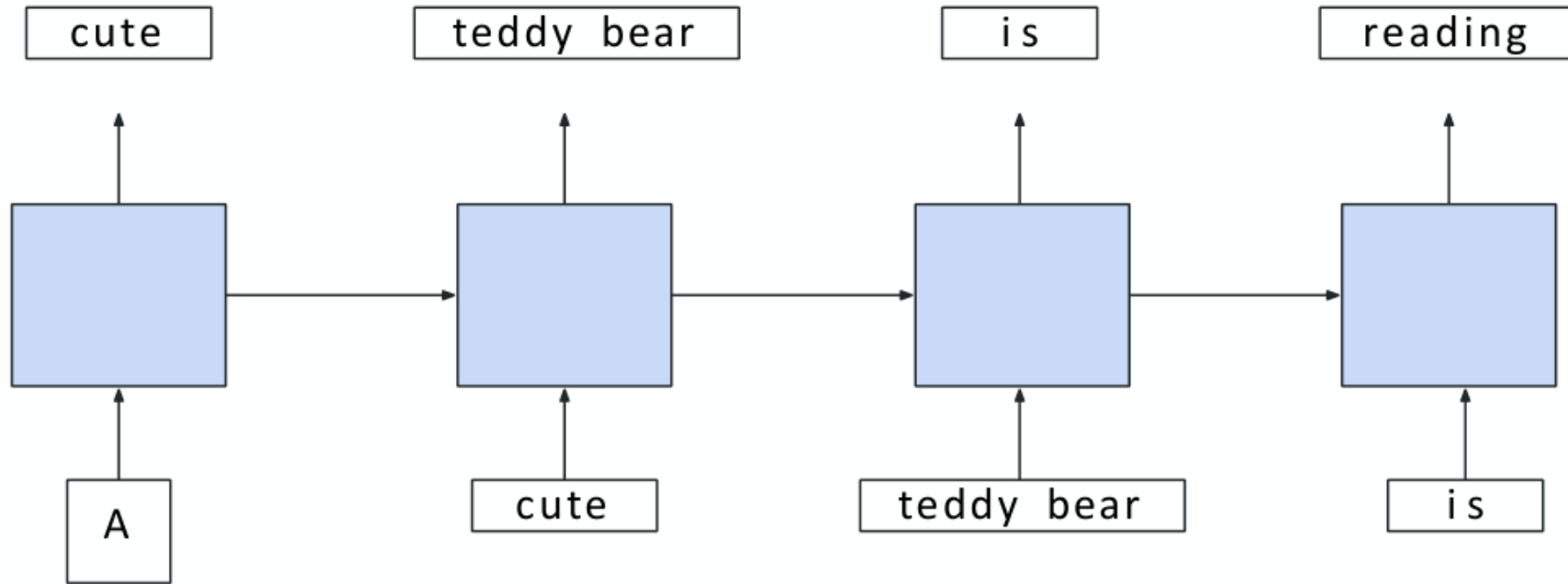
cute

teddy bear

Recurrent Neural Networks

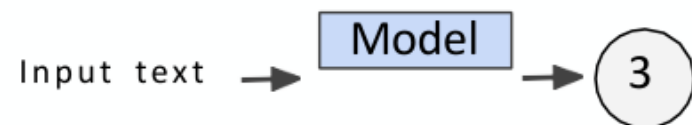


Recurrent Neural Networks

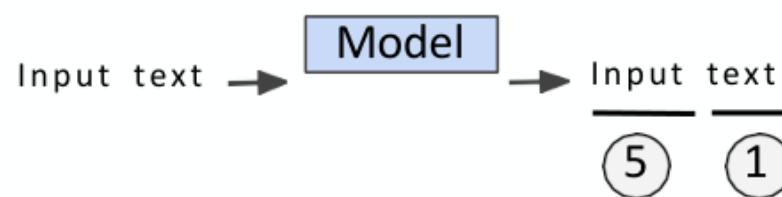


Recurrent Neural Networks

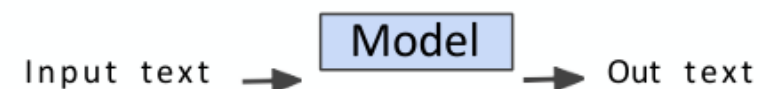
Classification



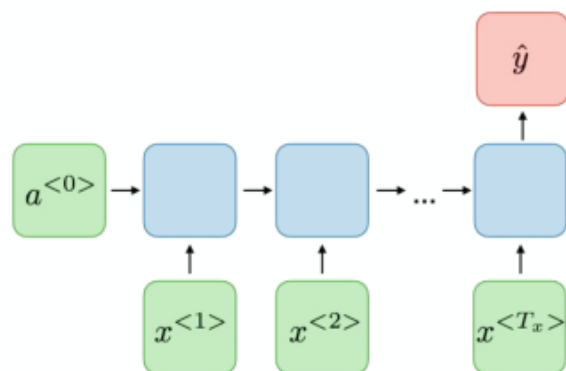
“Multi”-classification



Generation

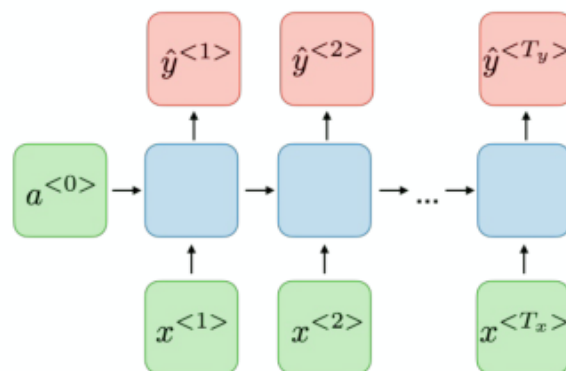


Sentiment



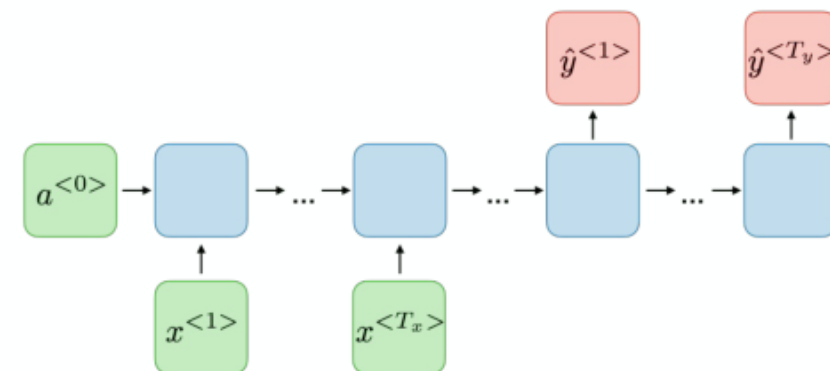
Opinion

Tags



Text

Translation



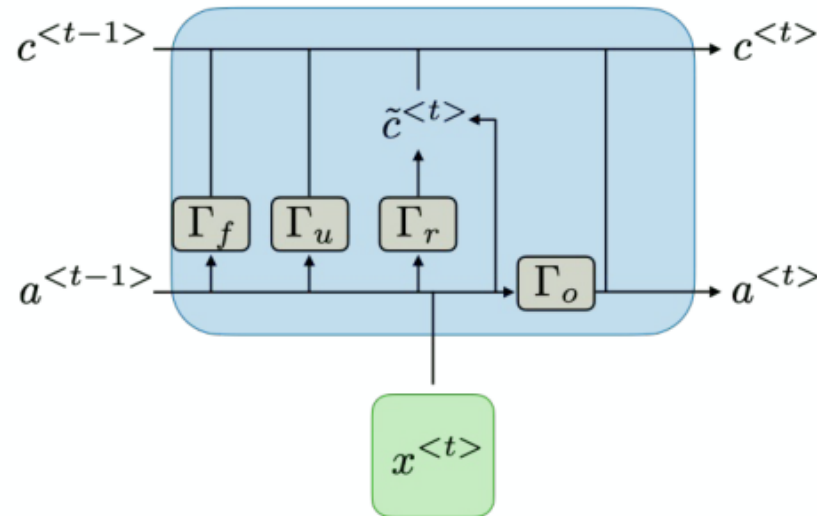
Source

Long Short-Term Memory (LSTM)

Overview

- Introduced in “[Long short-term memory](#)” (1997)
- Uses a more structured approach in the cell's hidden state

General form



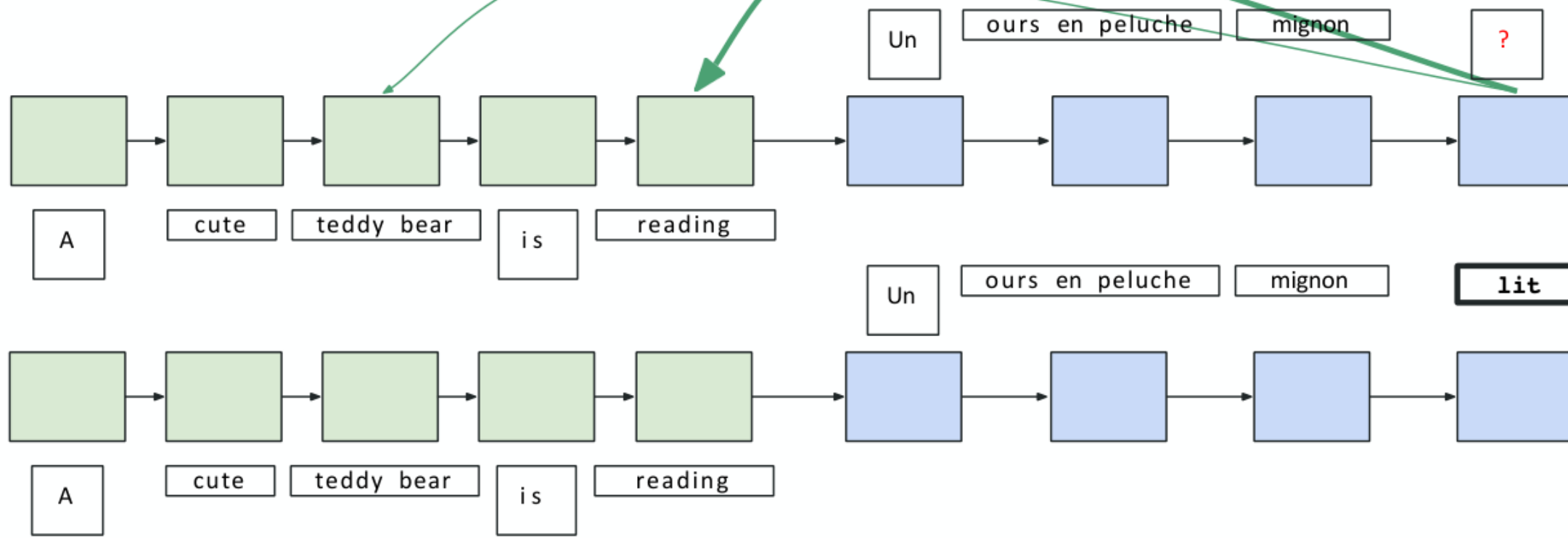
Summary of Main Methods

Method	Pros	Cons
Word2vec e.g. CBOW, Skip-gram	• Very simple, yet powerful • Intuitive embeddings	• Word order does not count • Embeddings not context aware
Recurrent Neural Networks e.g. traditional RNN, LSTM	• Word order matters • State-of-the-art results	• Vanishing gradient problem • Slow computations

History of Attention

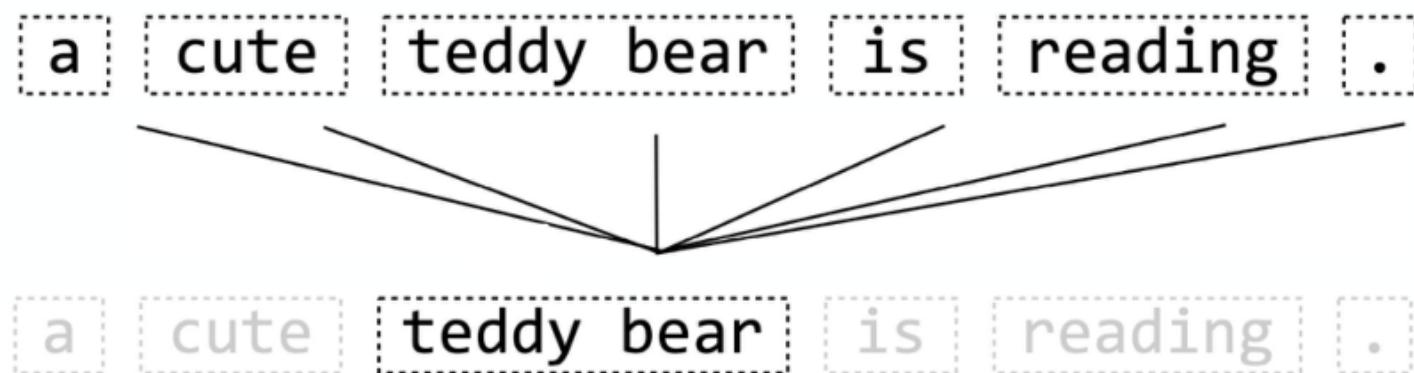
Introduced in 2014

- Translation tasks had a real issue with long-term dependencies
- Seq2seq unable to "remember" what input sentence was saying



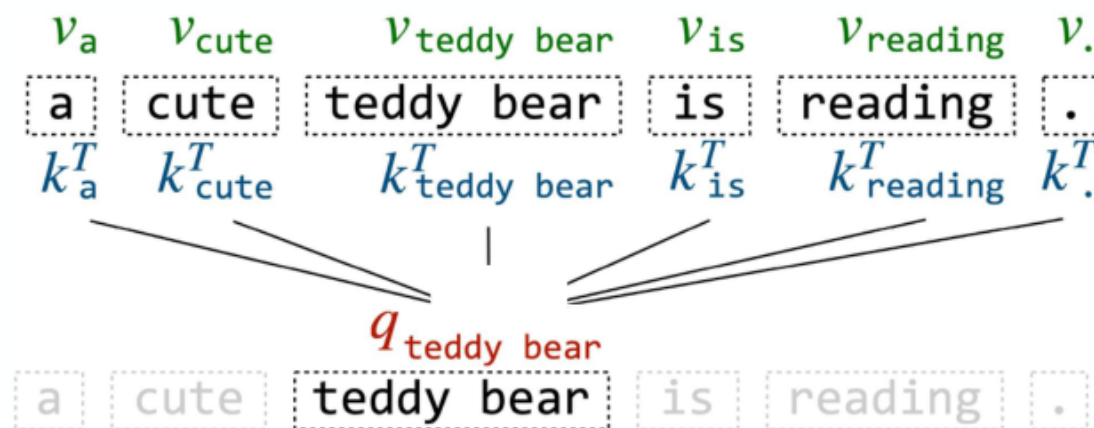
Overview of Transformer

- Introduced in the 2017 paper "[Attention is All You Need](#)"
- Relies on the **self-attention** mechanism
- State-of-the-art results on machine translation tasks



Attention Mechanism

- **Query (q):** "teddy bear" sends a question to every other word asking "how relevant are you to me?"
- **Key (k^T):** Each word advertises its identity; the dot product between Query and Key produces the attention score measuring relevance
- **Value (v):** Carries the actual information to pass forward — words with higher attention scores contribute more of their Value to the final output

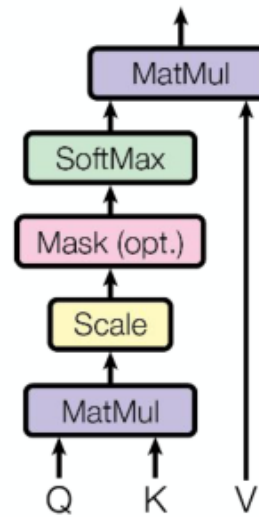


Attention Mechanism

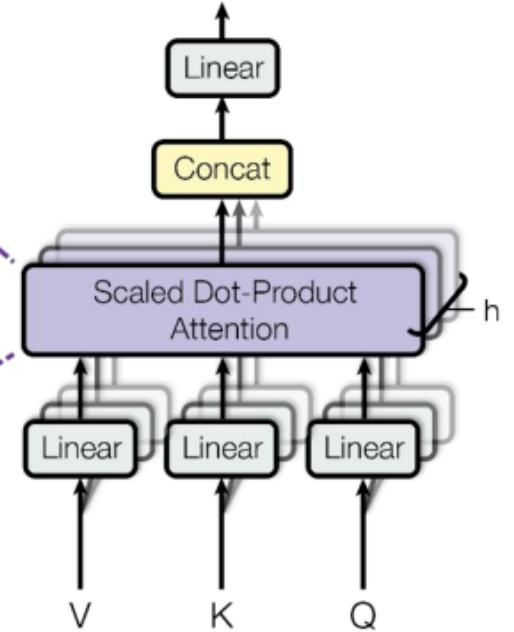
- Efficient computations with matrices:

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Scaled Dot-Product Attention

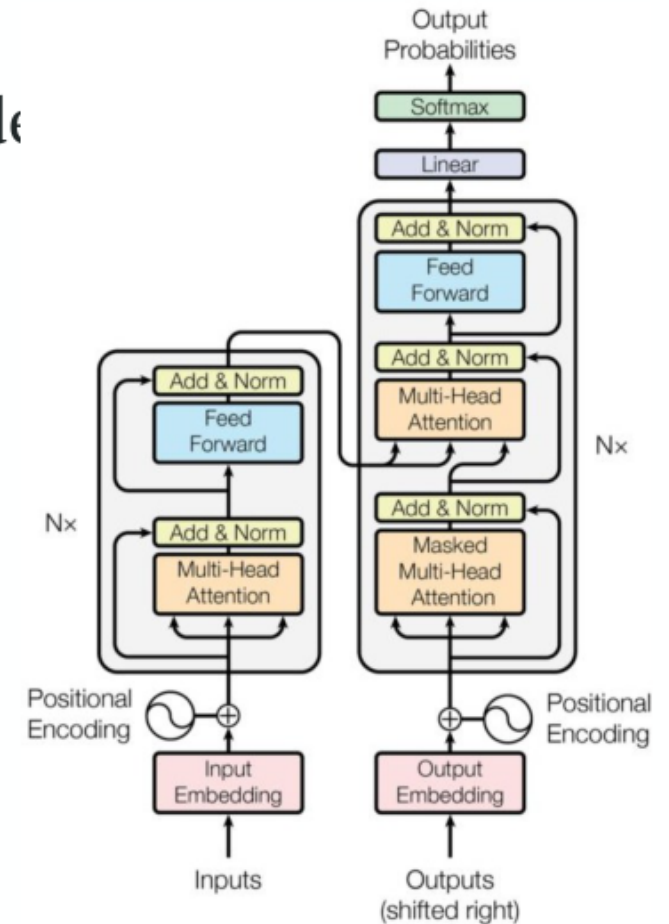


Multi-Head Attention



Attention Mechanism

- **Multi-Head Attention layer (MHA)**
 - Self-attention (Encoder-Encoder, Decoder-Decoder)
 - Encoder-Decoder attention layer
- Feed Forward Neural Network (FFNN)
- Positional Encoding (PE)



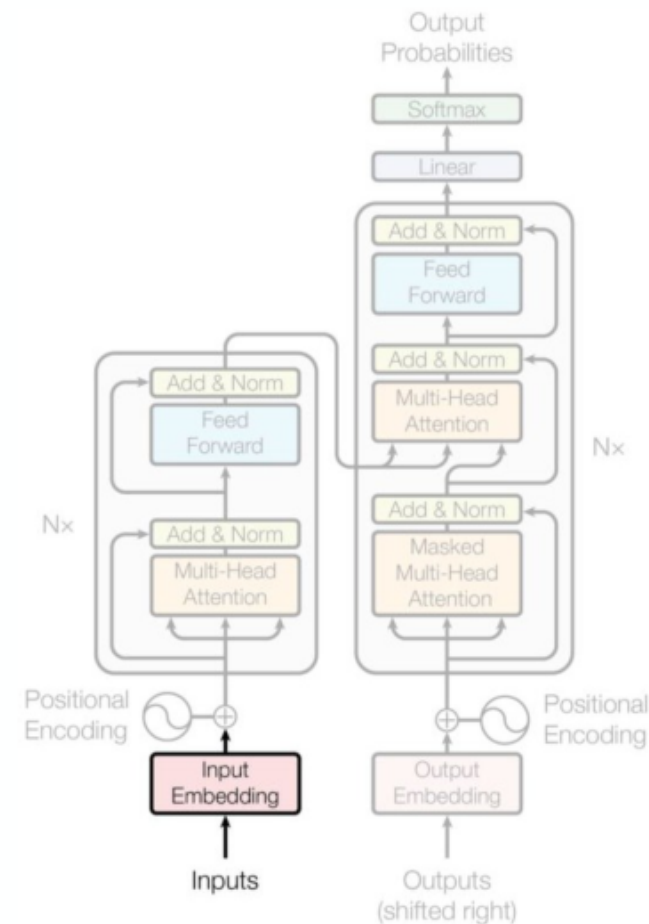
Input

Overview

- Text is "tokenized"
- Learned embeddings for tokens

Parameters

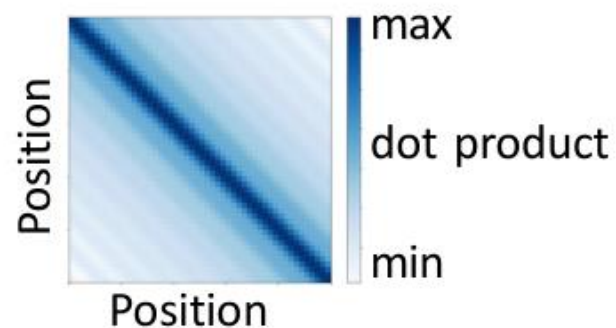
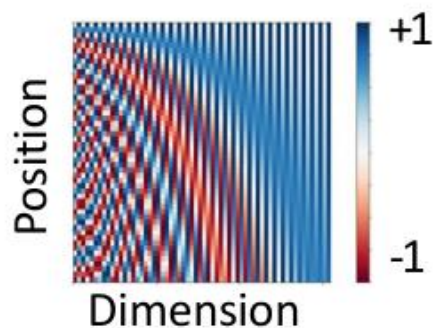
- V : vocabulary size
- d_{model} : embedding dimensions



Positioning Encoding

Idea:

- Add **position information** to inputs
- Can be either learned or hardcoded



- Goal: let model understand relative input position

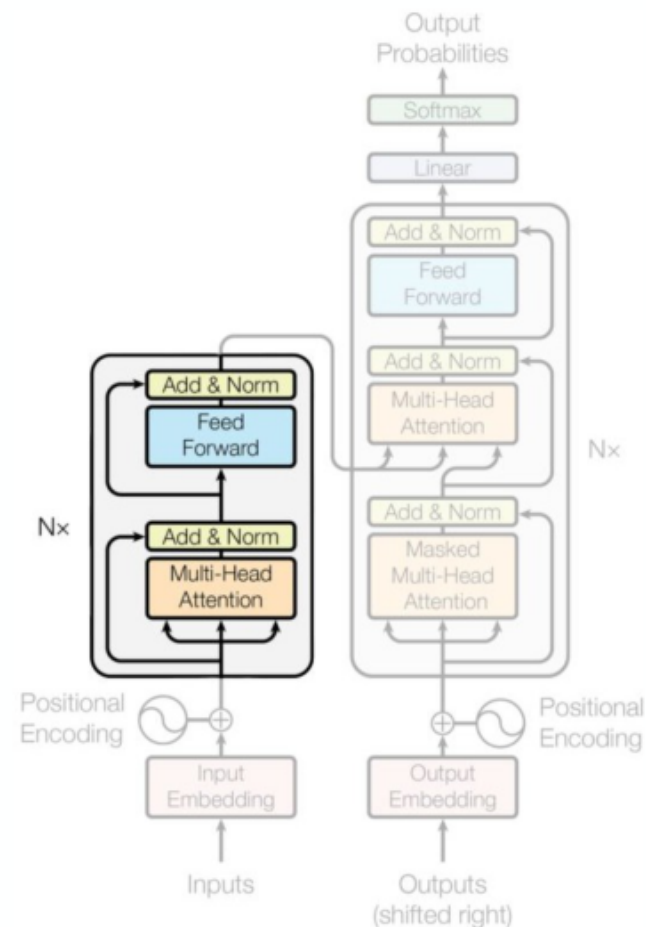
Encoder

Overview

- Encoder-Encoder attention / self-attention
- Feed Forward Neural Network
- Normalization layer

Parameters

- N : layers stacked
- h : number of attention heads
- $d_{FF}, d_{key}, d_{value}$: sub-layer dimension
- d_{model} : embedding dimensions



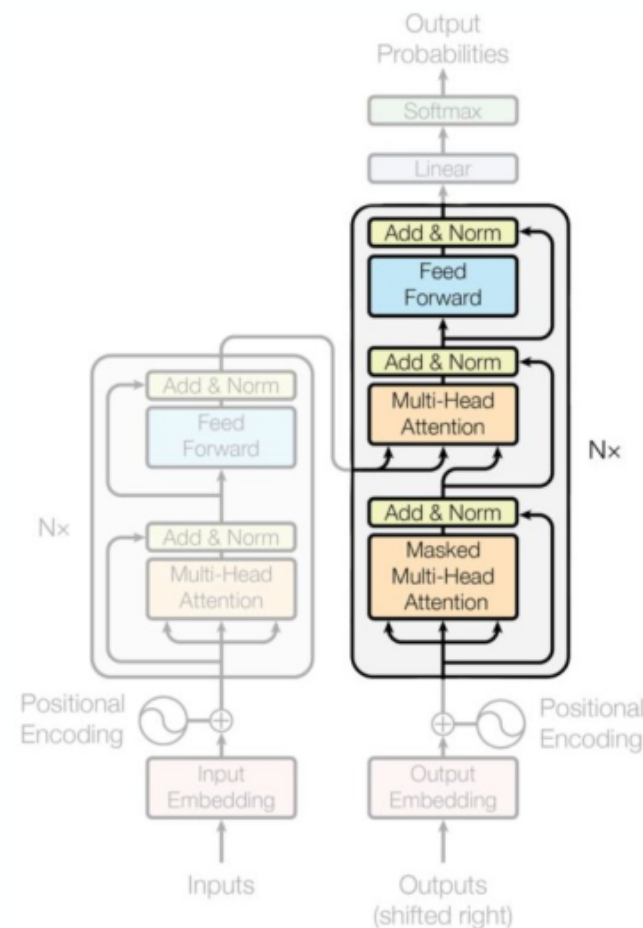
Decoder

Overview

- Decoder-Decoder attention / self-attention
- Encoder-Decoder attention
- Feed Forward Neural Network
- Normalization layer

Parameters

- N: layers stacked
- h: number of attention heads
- d_{FF} , d_{key} , d_{value} : sub-layer dimension
- d_{model} : embedding dimensions



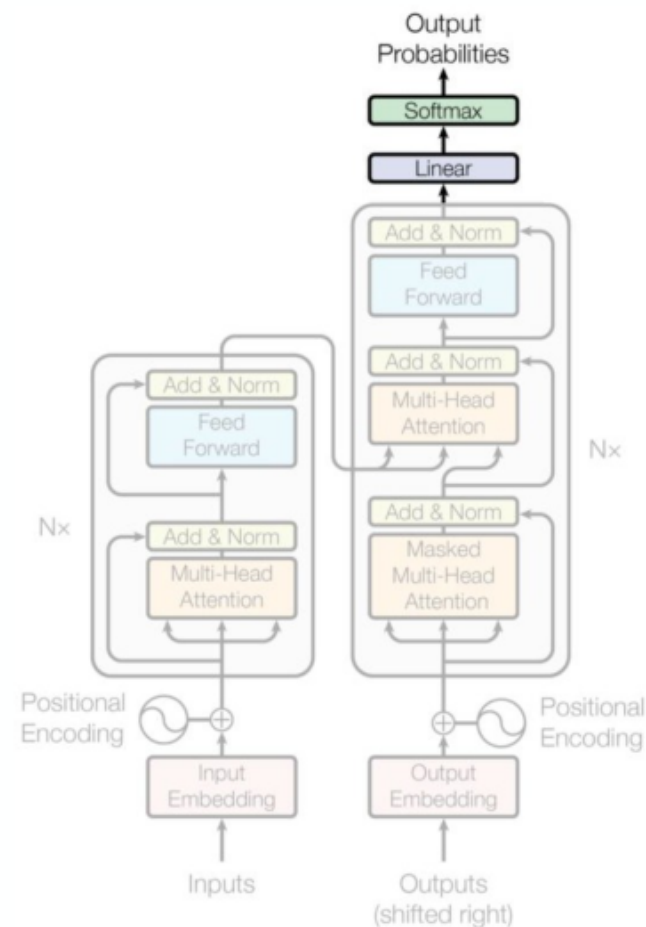
Output

Overview

- Linear projection
- Classification problem that outputs probability of belonging to a class, where class = word

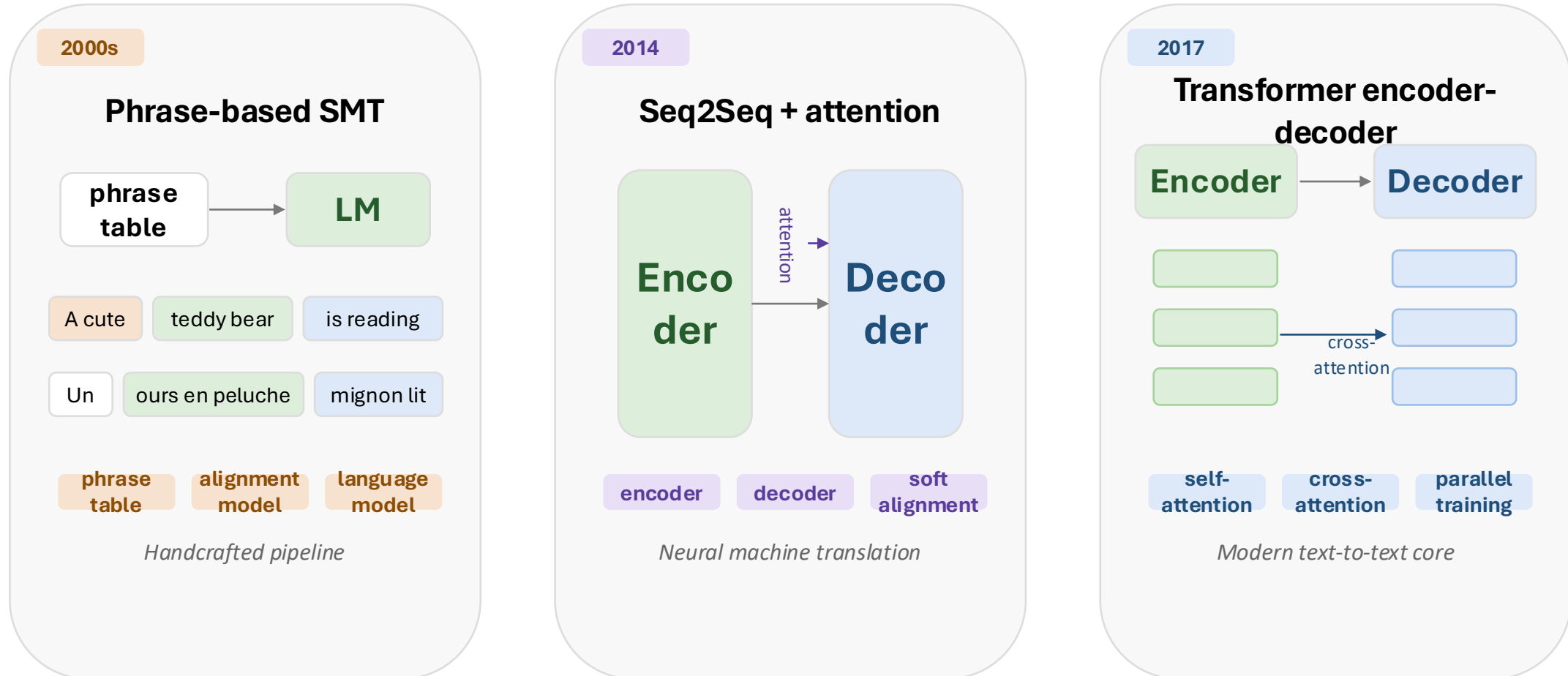
Parameters

- V : vocabulary size
- d_{model} : embedding dimensions



Before Decoder-Only LLMs, Translation Was a Specialized Pipeline

- Phrase-based statistical machine translation used phrase tables and a separate language model.
- Seq2Seq with attention then learned an encoder-decoder mapping end to end and handled long-range dependencies better.
- The 2017 Transformer kept the encoder-decoder structure but replaced recurrence with attention.



Translation history moved from hand-engineered phrase systems to end-to-end encoder-decoder models.

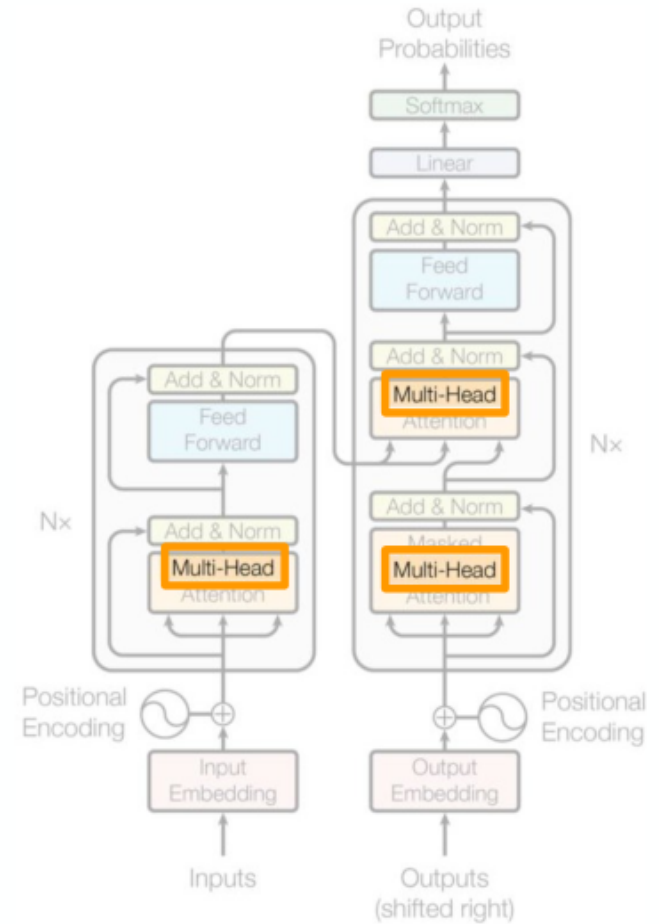
Multi-Head Attention

Idea:

- Run **multiple** self-attention layers in parallel

Benefits:

- Enables the model to capture different attention features in parallel
- Comparison: **multiple** filters of a convolutional layer in computer vision



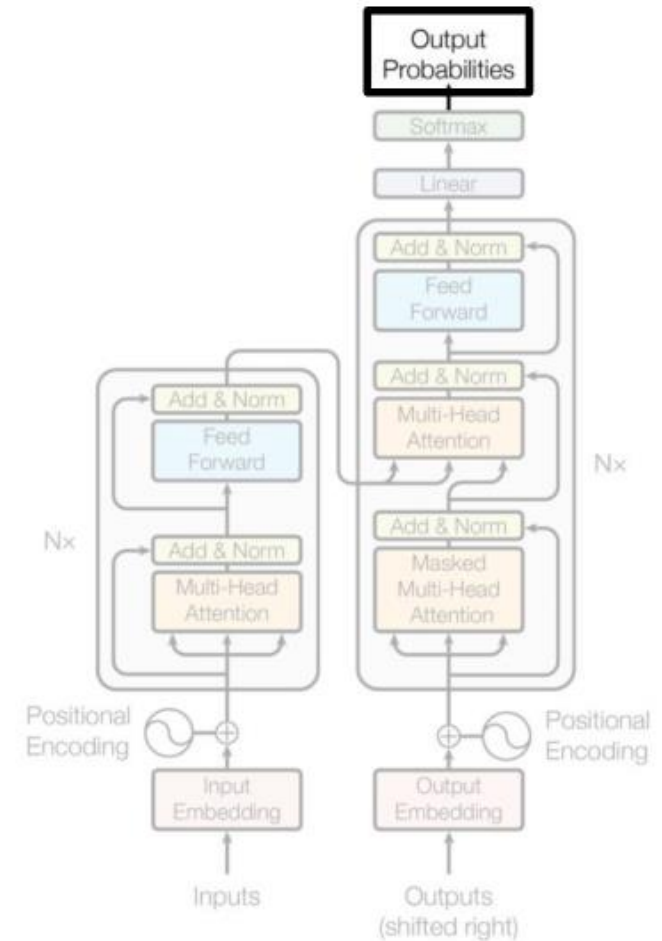
Computational Tricks: Label smoothing

Idea:

- Overconfidence is bad
- Introduce **noise** in true labels

Benefits:

- General technique that prevents overfitting
- Improves accuracy and BLEU score



Q&A

Any question?